Midterm Review

Announcements

- HW4 was due last night

- Midterm next class

General Exam Information

- 80 minutes
- Closed book. Closed notes.
- Short answer
- It will be a long exam. Use it as an opportunity to show everything that you know.

Topics

Foundations:

- Undecidability and what it means for program analysis
- Soundness, completeness, and the tradeoffs between them

Program Representations:

- Reading and writing JVM bytecode
- Exception tables and try-catch in bytecode
- ASTs

Topics

Testing and Coverage:

- Statement, branch, and path coverage, and their limitations
- Writing JUnit tests

Test oracles:

- Test prefix vs. oracle
- Safety, functional, and regression oracles
- Generating oracles: specification mining from docs, IR (Jaccard), neural, grammar-constrained (TOGA)
- How regression oracles are derived from mutants

Topics

Random testing:

- Randoop’s feedback-directed generation
- Classifying sequences: illegal, redundant, contract violation, add to components
- What contracts can and can’t catch

Bytecode and ASTs

```
static int f(int, int);
  Code:
     0: iload_0
     1: istore_2
     2: iload_1
     3: iload_0
     4: if_icmple     9
     7: iload_1
     8: istore_2
     9: iload_2
    10: iconst_2
    11: imul
    12: ireturn
```

(a) What do f(4,7) and f(9,2) return

(b) Write java source that compiles to this bytecode

Bytecode and ASTs

```
static int f(int a, int b) {
 0: iload_0
 1: iload_1
 2: idiv
 3: istore_2
 4: goto          10
 7: astore_3
 8: iconst_0
 9: istore_2
10: iload_2
11: ifle          18
14: iload_2
15: iconst_2
16: imul
17: istore_2
18: iload_2
19: ireturn
Exception table:
   from    to  target type
       0     4     7   Class
java/lang/ArithmeticException
```

(a) What is the return value of `f(7, 2)`?

(b) Write Java source that compiles to this bytecode.

Exceptions and Exception table

In source,

- `try { ... }` marks code that might throw
- `catch (T e) { ... }` runs if something of type T (or a subclass) is thrown inside the try
- If no catch matches, the exception propagates to the caller

In bytecode,

- There's no "try" instruction. The try block is just normal code.
- Bytecode includes an **exception table.** Each row says "if an exception of type T is thrown between offsets `from` and `to`, jump to `target`."
- astore saves the exception in e

from	to	target	type
0	4	7	java/lang/ArithmeticException

Binary vs Unary comparisons

We saw two types of comparisons in our examples:

- if_icmple (eq, neq, lt, ge, gt)
  - pops two values off the stack
  - value1 <= value2 (where value2 is the top of the stack)
- ifle (eq, neq, lt, ge, gt)
  - Pops one value off the stack
  - Compares it to 0
  - value1 <= 0

The condition is flipped from the source. If it is true, we jump. Otherwise, we enter the condition body.

Bytecode and ASTs

(c) Draw the AST for return (a + b) * c;

Coverage and Mutants

```
static int sign(int x) {

    if (x > 0) return 1;

    if (x < 0) return -1;

    return 0;

}

T1: assertEquals(1, sign(5));
T2: assertTrue(sign(-3) < 0);
```

(a) What are the statement coverage and branch coverage of the suite?

(b) For each mutant, say whether the suite kills it:

- M1: x > 0 becomes x >= 0
- M2: return -1 becomes return -2
- M3: return 0 becomes return 1
- M4: x < 0 becomes x > 0

(c) What is the mutation score? Add the fewest assertions needed to kill every surviving mutant.

Oracles

(a) Here is a grammar of assertions:

```
<oracle> ::= assertTrue(<e>)

<e>      ::= x | !<e> | <e> && <e>
```

Write a derivation for `assertTrue(!x && x)`.

Is the language finite? Why or why not?

Oracles

(b) A regression oracle generator runs this prefix:

```
Stack s = new Stack();

s.push(4);

s.push(9);

int t = s.pop();
```

On the original program, t = 9 and s.size() = 1.

On a mutant where pop returns the bottom element, t = 4 and s.size() = 1.

Which assertion(s) does the generator add, and why?

Randomized Testing with Randoop

```
class Box {

    int v;

    Box(int v) {

        if (v > 100) throw new IllegalArgumentException();

        this.v = v;

    }

    void set(int n) { v = n; }

    public boolean equals(Object o) {

        return o instanceof Box && ((Box) o).v == v;

    }

    public int hashCode() { return 100 / v; }

}
```

Randoop checks the contract “o.hashCode() throws no exception.” It generates these sequences in order, starting with no Box objects. Classify each as illegal, contract violation, redundant, or add to components.

1. Box b = new Box(150);
2. Box b = new Box(3);
3. Box b = new Box(3);
4. Box b = new Box(3); b.set(0);
5. Box b = new Box(5);

Jaccard

A streaming service recommends the movie whose tags are most similar to the one you just watched, using Jaccard similarity.

You watched: Q = {action, sci-fi, space, robots}

Corpus:

A: {action, sci-fi, space, aliens}

B: {sci-fi, robots}

C: {action, comedy, robots, space, romance, musical, holiday}

(a) Compute J(Q, A), J(Q, B), and J(Q, C). Which movie is recommended?
(b) Every one of B's tags appears in Q, but B doesn't score higher than A. Why not?

Summary

- Midterm next class!
- Skills to practice:
  - Trace bytecode by hand
  - Draw ASTs for expressions
  - Compute statement and branch coverage and decide whether a test kills a mutant
  - Compute jaccard similarity
- Concepts to understand:
  - Randoop’s algorithm
  - Why coverage alone does not constitute a bug-finding test suite
  - Test prefix vs oracle
  - How regression oracles are generated
  - Approaches to generate oracles (IR, grammar constrained ranking)